

Efficient DDoS Detection and Mitigation in Cloud Data Centers Using eBPF and XDP

Ziyue Chen^{*†}, He Kong^{*†}, Shuai Ding^{*†}, Quanfeng Lv^{*†}, and Guo Wei^{*†}

^{*}Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences

[†]School of Cyber Security, University of Chinese Academy of Sciences

Email: chenzyue211@mailsucas.ac.cn

Abstract—With the widespread adoption of cloud-native architectures, data centers are increasingly facing severe security challenges, particularly in defending against distributed denial-of-service (DDoS) attacks. The complex interdependencies between microservices allow the impact of a DDoS attack to spread rapidly, significantly degrading overall server performance. Traditional DDoS defense strategies often struggle to effectively counter attacks within data centers, particularly those targeting east-west traffic. To address this issue, this paper proposes a fast and efficient DDoS detection and mitigation mechanism specifically tailored for data center environments using extended Berkeley Packet Filter (eBPF) and eXpress Data Path (XDP) technologies. By leveraging eBPF, we achieve kernel-level, non-intrusive data collection while maintaining flow state tracking, thereby reducing communication overhead. Additionally, we trained an XGBoost classification model to create DDoS traffic filtering rules specifically designed for data center scenarios. Combined with XDP technology, we implemented low-latency malicious traffic detection at the entry point of the network protocol stack, allowing for the immediate filtering of attack traffic and thereby mitigating the impact of DDoS attacks. Experimental results demonstrate that, compared to traditional tools, our approach not only achieves high accuracy but also significantly enhances system performance.

Index Terms—eBPF, XDP, DDoS attack, machine learning.

I. INTRODUCTION

In recent years, cloud computing has rapidly evolved due to its distinctive features such as on-demand self-service, resource pooling, and observability [1], [2]. To further optimize application performance, availability, and scalability, cloud-native technologies have emerged [3]. However, the distributed nature of cloud-native deployments has introduced significant network security challenges, creating a vast attack surface for potential threats like DDoS attacks [4], [5]. Statistics show that traffic between different servers within data centers accounts for more than 75% of the total traffic [6]. Therefore, it is crucial to pay close attention to attack traffic resulting from tenant-to-tenant flooding within data center environments. To effectively analyze data flows in the resource-constrained context of data centers, two key challenges must be addressed. The first challenge is how to increase the data sampling frequency and detection efficiency during the detection phase while minimizing resource consumption. Traditional flow-based detection tools require monitoring the entire lifecycle state of data flows, which can lead to significant redundancy when parsing packets. The second challenge is how to reduce

system resource consumption during the attack mitigation phase when matching a large number of rules.

To address the aforementioned challenges, this paper proposes a novel DDoS detection and mitigation solution based on extended Berkeley Packet Filter (eBPF) and eXpress Data Path (XDP) technologies. This approach aims to reduce CPU and bandwidth resource consumption in data centers while enhancing security and reliability. Our solution collects and updates statistical information within eBPF Maps during specific time windows in the kernel, effectively reducing communication overhead and resource consumption between the kernel and user space. The use of XDP at the network interface for filtering abnormal traffic significantly improves attack mitigation efficiency. Additionally, this system operates transparently to user-space network applications, and the use of Cilium-eBPF's CO-RE (Compile Once – Run Everywhere) method further enhances the portability of the programs, making it highly suitable for distributed architectures in data centers.

The experimental results demonstrate that under full-load conditions, the CPU usage of the eBPF-based kernel data collection approach is approximately 16.7% of that of Wireshark. Additionally, memory usage and average response latency were reduced by 11.0% and 11.4%, respectively. On the extended CIC-DDoS2019 dataset [7], the XGBoost model achieved an impressive accuracy of 99.0%. Compared to the iptables approach, the XDP solution showed a significant 40.6% improvement in throughput and a substantial 1.23-fold optimization in CPU usage.

Our main contributions are summarized as follows:

- We present an eBPF/XDP-based DDoS detection and mitigation scheme for data center environments, capable of efficiently identifying and blocking DDoS attacks with minimal overhead in resource-constrained settings.
- We designed a prototype system based on eBPF, which effectively reduces communication costs and resource consumption for storing information during interactions, enabling efficient network data flow feature collection.
- We evaluated the runtime performance consumption of our system and conducted extensive comparative experiments with open-source tools. The results show that our system consumes significantly fewer resources when collecting data compared to Wireshark, which processes data in user space; it also exhibits lower performance

overhead during packet filtering than iptables using linear matching rules; overall, our system demonstrates slightly better performance than Snort during operation.

The rest of this paper is structured as follows. Section II reviews related work, Section III provides design details, Section IV presents experimental results, and Section V concludes the paper.

II. RELATED WORK

A. DDoS Attack Detection Method

DDoS attack detection primarily relies on two techniques: signature-based detection and anomaly-based detection. Signature-based detection utilizes a database of known attack patterns but faces challenges in identifying evolved attacks and zero-day threats. For instance, Bro (now known as Zeek) passively monitors network connections, simplifying network traffic into higher-level events to detect intrusions [8]. Snort, as a lightweight network intrusion detection system, enables real-time traffic analysis [9].

Anomaly-based detection, on the other hand, focuses on identifying patterns that deviate from expected behavior [10]. Before detection can occur, a baseline profile of network behavior during non-attack periods must be created, often using statistical methods or machine learning techniques [11]–[13]. For example, Shieh et al. employed a bidirectional LSTM model combined with a Gaussian mixture model to detect and classify six different types of DDoS attacks, achieving an accuracy rate of 98% [14]. Varghese et al. proposed a statistical anomaly detection algorithm implemented in the software-defined networking (SDN) data plane, enabling near real-time DDoS attack detection [15].

Detecting DDoS attacks in the high-speed networks of data centers requires rapid packet processing to minimize server strain. However, detection methods that rely on traffic statistics can introduce additional costs and performance issues to the network [16]. Recent research has begun to focus on how to achieve secure and high-performance data collection and analysis at the host level.

B. DDoS Attack Mitigation Method

Once malicious traffic is detected, the most commonly used response mechanism is traffic filtering. Al-Haidari et al. proposed a mitigation technique called EDos-Shield, which has a virtual firewall and a verifier cloud node [17]. Masood et al. introduced a mitigation technique that offers dual controls. Access control limits the number of clients, while congestion control prioritizes permitted clients based on learned browsing behavior [18].

Traditional DDoS mitigation methods often impose significant overhead on clusters and may negatively impact the user experience for legitimate users [19]. To address these issues, recent studies have introduced XDP to mitigate high-volume DDoS attacks, enhancing the efficiency of traffic filtering.

C. DDoS Attack Detection and Mitigation Methods Based on eBPF and XDP

Existing research on the application of eBPF and XDP to DDoS can be divided into two categories: (1) Focus on the use of eBPF in complex network structures, facilitating the development of various intrusion detection strategies [20]. (2) Focus on server-side DDoS attack mitigation, to bypass the kernel and filter out all detailed information on the user side, minimizing the overhead on the Linux network stack [21]. However, current eBPF or XDP-based solutions are not widely applied in traditional network applications like firewalls or other data plane services [22]. Most existing approaches focus on statistical anomaly detection, which is prone to false positives when normal network traffic fluctuates. [23] [24].

We propose a novel DDoS detection and mitigation solution based on eBPF and XDP. This approach maintains and aggregates contextual flow information within time windows in the eBPF Map, addressing the limitations of traditional eBPF-based anomaly detection, which often focuses solely on packet characteristics and results in lower detection efficiency. Additionally, it overcomes the drawbacks of traditional flow-based detection methods that require tracking complete data flows, which consume significant resources and lack real-time efficiency. By integrating with XDP, the proposed method performs rule matching at the earliest level of the Linux network stack—immediately after the driver's RX queue—using the longest prefix match algorithm. This results in lower resource consumption and higher efficiency compared to traditional methods.

III. DDOS ATTACK DETECTION AND MITIGATION BASED ON EBPF/XDP

We designed the overall architecture of a DDoS attack detection and mitigation system based on eBPF/XDP, which can be divided into three main components: the data collection module, the rule generation module, and the packet filtering module, as shown in Figure 1. The following sections of this paper will provide a detailed description of the design of each module.

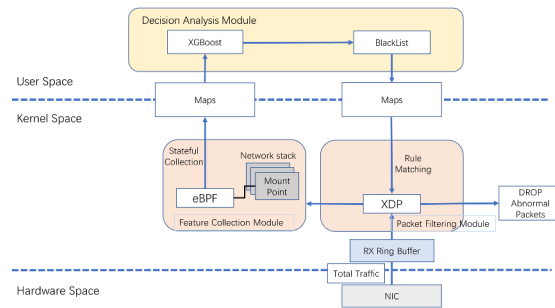


Fig. 1: Overall System Architecture.

A. eBPF-based Data Collection

The data collection focuses on the rapid aggregation and statistical analysis of certain flow metrics. To implement a lightweight data collection module for flow capture and filtering, we leverage eBPF for stateful data collection within the kernel. The data collection program is attached to network interfaces, enabling real-time aggregation and analysis of flow data within eBPF Maps. The monitoring points are set at xdp, ip_rcv, tcp_rcv, and udp_rcv. When the functions at these monitoring points are triggered, the corresponding eBPF program is executed. The process is as follows: Once the data collection module is deployed on a network port, each time a packet triggers the attached kernel function, the kernel program begins execution to process the packet. The flow of packets through this module is illustrated in Figure 2.

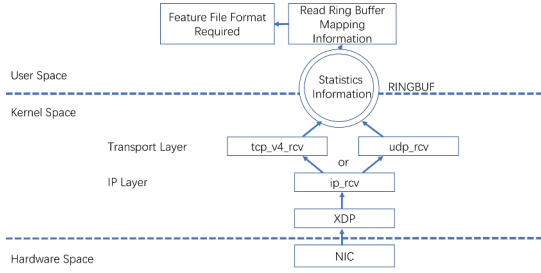


Fig. 2: Flowchart of the Data Collection Program.

The information gathered by our approach includes network statistics within a time window, such as the transmission rate of network flows classified by five-tuples, counts of specific flag bits, and average packet size. To reduce system overhead during data interaction between kernel and user space, the kernel data collection code preprocesses the collected data. The data is aggregated and calculated based on five-tuples within a time window, further minimizing the cost of data transmission from the kernel to user space. The extracted features are listed in Table I, excluding attributes that do not contribute to the model's generalization, such as IP addresses, link-layer encapsulation types, and application-layer properties.

B. XGBoost-based Anomaly Detection

To capture as many attack types as possible, this paper extends the CIC DDoS 2019 dataset to train an anomaly classification model. The primary work includes expanding the dataset with additional application-layer attack types and normal traffic data. During the design of the data collection, we referred to the feature importance rankings from the training process of the original dataset when selecting data features.

After completing the model training, the trained XGBoost model was exported in the PMML (Predictive Model Markup Language) format. This allows the model to be directly read within the eBPF user-space program. The source IP addresses

TABLE I: Table of statistical feature information

Feature Name	Description
Timestamp	The time interval between the current packet and the first packet in the flow, with the first packet being 0
Source IP	Source IP address
Source Port	Source port
Destination Port	Destination port
Protocol	Protocol type
Flow Packets/s	Number of packets per second
Flow Bytes/s	Number of bytes per second
ACK Flag Count	Number of times the ACK control bit is set to 1
SYN Flag Count	Number of times the SYN control bit is set to 1
RST Flag Count	Number of times the RST control bit is set to 1
URG Flag Count	Number of times the URG control bit is set to 1
CWR Flag Count	Number of times the CWR control bit is set to 1
Packet Length Mean	Average packet length
Min Packet Length	Minimum packet length

of anomalous packets will be output to a blacklist for subsequent processing.

C. XDP-Based Packet Filtering

XDP can directly inspect network packets at the lowest level of the kernel space, and its event-driven execution mode ensures that resources are consumed only when necessary. The design of this part is shown in Figure 3. In the packet filtering module, the blacklist constructed by the anomaly detection module is batch-updated into a map. We use the BPF_MAP_TYPE_LPM_TRIE map type. XDP parses packets and uses the LPM algorithm to match against the blacklist, immediately discarding packets from malicious sources, while packets that pass the check are redirected to the next program in the pipeline—feature extraction—for further anomaly detection. When the XDP program is enabled, it performs packet inspection immediately after the packet is received by the network interface card and before it enters the kernel protocol stack.

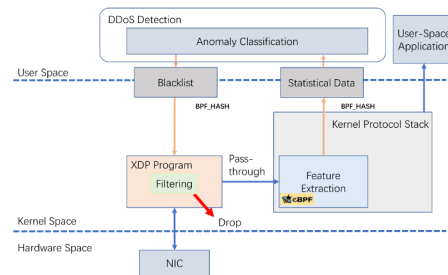


Fig. 3: Implementation of Attack Mitigation.

IV. EXPERIMENTAL SETTINGS

A. Experimental Platform

The entire experimental platform is deployed on a local Kubernetes cluster consisting of six machines. Each machine is equipped with an Intel(R) Xeon(R) CPU E5-2660 0 @ 2.20GHz, 4 vCPUs, 8GB of RAM, running Ubuntu 22.04

with a kernel version of 5.15.0-33-lowlatency, and Kubernetes version 1.23.4. We deployed the social network project from the open-source benchmark suite DeathStarBench. The test setup in the cloud environment is illustrated in Figure 4. On Host 1, two containers, and on Host 2, one container is defined as the attack nodes, where hping3 is used to simulate DDoS attacks. The remaining containers generate normal traffic, which is aggregated and monitored as it flows into Host 3. Host 3 serves as the target machine, where the system for DDoS attack traffic detection and mitigation is deployed.

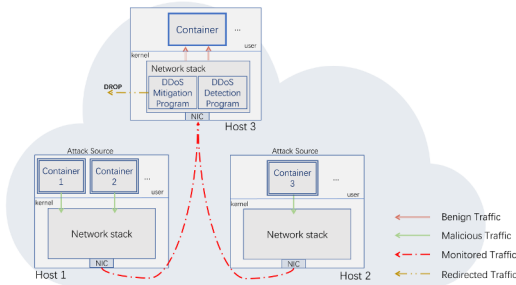


Fig. 4: Experimental environment diagram.

B. Performance of eBPF

To evaluate the impact of eBPF processing on container network performance, we tested the effects of our approach and Wireshark on key network performance indicators—throughput, latency, CPU usage, and memory utilization—while analyzing high-volume data. The baseline configuration, which did not involve any packet capture or filtering, was used as a reference.

The impact of various packet capture tools on system TCP throughput is shown in Figure 5(a). Throughput was measured every 50 seconds for Wireshark, eBPF, and the baseline. The results indicate that using eBPF for packet capture has a significantly smaller impact on system performance compared to Wireshark. Furthermore, eBPF's impact on system throughput is minimal compared to the system's state when no packet capture is performed.

The specific results for CPU usage are shown in Figure 5(b). We used hping3 to perform a SYN Flood attack at a bandwidth rate of approximately 1.92 Gbps for 120 seconds. It is evident that eBPF maintains stable CPU usage under full load, with almost no additional CPU overhead compared to the system state without packet capture. In contrast, Wireshark consumes about six times more CPU resources than eBPF.

In a similar setup, we compared the memory usage of eBPF and Wireshark during packet capture. The results are shown in Figure 5(c), indicate that eBPF improves memory utilization by approximately 11% compared to Wireshark.

For network latency impact experiments, we used the ethr tool to simulate network traffic, as shown in Figure 5(d). Wireshark analyzes packets by copying them from the kernel buffer to user space, which substantially impacts TCP

transmission latency. In contrast, eBPF performs sampling within the kernel and captures packets only for exceptional cases, resulting in minimal impact on network latency, with performance remaining nearly unchanged compared to when no packet capture is performed.

C. Performance of XDP

We conducted performance tests comparing XDP with the traditional Linux kernel firewall iptables, focusing on how the number of rules affects performance. Throughput was compared using iperf3, evaluating the performance impact of packet filtering with iptables versus packet filtering with XDP using eBPF code. The results are shown in Figure 6(a). It is evident that the impact of iptables on system throughput is significantly greater than that of XDP. With increasing rule numbers, XDP demonstrates a performance improvement of 82.0% when the rule count reaches 512. This indicates that XDP can handle packets more efficiently as the number of rules increases, leading to better network performance and faster data transmission speeds. Using hping3, we simulated a SYN flood attack with approximately 1.92 Gbps from six spoofed IPs. As Figure 6(b) shows, as the number of rules increased, the performance impact of XDP on CPU usage was almost negligible compared to iptables. In contrast, with more rules, iptables require substantial CPU resources for linear matching, causing a rapid increase in CPU usage. When the rule count reached 512, the CPU utilization with XDP was only 44.8% of that with iptables. Considering the impact of memory overhead on cloud services, we compared the memory usage of XDP and iptables, as shown in Figure 6(c). The difference in memory usage is not very pronounced, which may be due to the memory management mechanisms affecting their usage rates similarly.

D. Performance of Anomaly Detection

For the selected deep learning and machine learning models, this paper evaluates model performance using the following metrics: accuracy, precision, recall, and F1 score, all derived from the confusion matrix.

We evaluated several commonly used machine learning and deep learning algorithms for detecting and identifying DDoS attacks. After model training, we compared different algorithms using the test set. The accuracy, precision, recall, and F1 score results for these models are shown in Figure 7(a). To further evaluate model performance, we also tested the normal and attack detection rates of different models, as shown in Figure 7(b). Here, Normal_Detect_Rate represents the detection rate of normal instances, and Atk_Detect_Rate represents the detection rate of attack instances. The normal detection rate is calculated as the proportion of undetected normal samples out of the total number of normal samples, while the attack detection rate is the proportion of undetected attack samples out of the total number of attack samples. The experimental results indicate that XGBoost achieves the best classification performance and is selected as the final machine learning model.

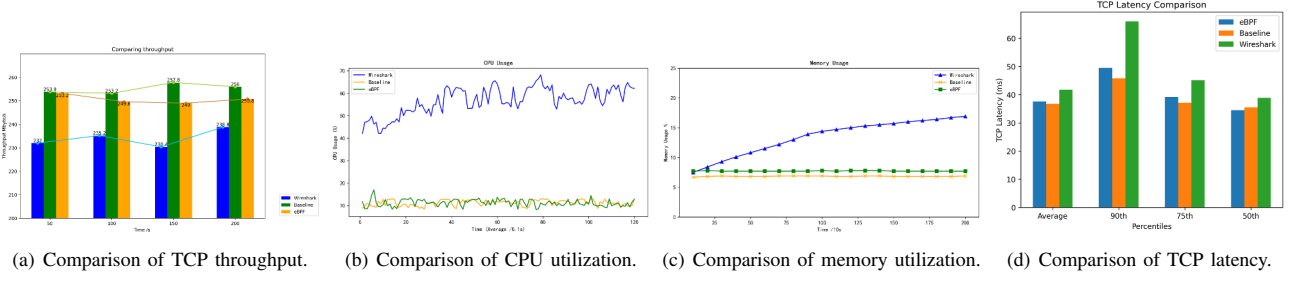


Fig. 5: Performance comparison between eBPF and Wireshark

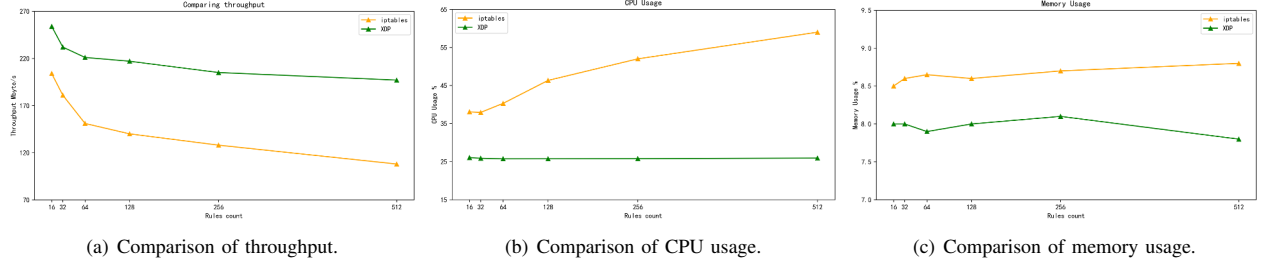


Fig. 6: Performance comparison between XDP and iptables

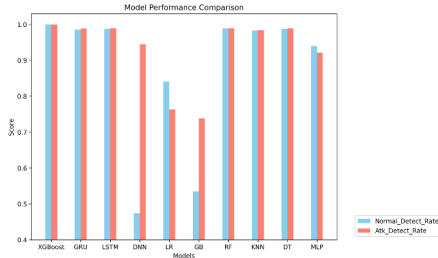
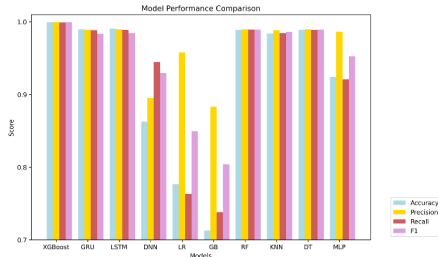


Fig. 7: Performance of Anomaly Detection

E. End-to-End Performance

Finally, we evaluate the overall performance of the system. In the experiment, Tcpreplay was used to replay the attack packets, with a packet transmission rate of 180,163 pps. The time taken to process a single packet for traffic information

statistics, anomaly detection, and packet filtering is shown in Table II. To evaluate the performance advantages of the proposed system in host-based DDoS detection and mitigation, a performance comparison was conducted with the open-source IDS tool Snort. In this experiment, the UDP Flood attack was launched on the attack container using hping3, and the rules deployed for Snort are listed in Table III.

TABLE II: Processing Time for a Single Data Packet

Process	eBPF	XGBoost	XDP
Processing Time (us)	12.68	11.96	5.72

During the experiment, the CPU usage, memory utilization, and throughput of the system and the Snort tool under different attack scales were collected, and the average values over 120 seconds were calculated. Figures 8(a), 8(b), 8(c) show the system resource consumption of this system and Snort under different attack intensities. The experimental results show that during the test, the resource consumption of this system was consistently lower than that of Snort, indicating that this system can operate more efficiently in the face of high-volume DDoS attacks. The poor performance of Snort is mainly due to its use of regular expressions to match the attacker's patterns in network traffic. The lower resource consumption of this system is mainly attributed to the use of eBPF technology, which has higher efficiency.

V. CONCLUSION

In this paper, we implemented an efficient network packet feature extraction and analysis method based on eBPF, which significantly improves the processing efficiency of network

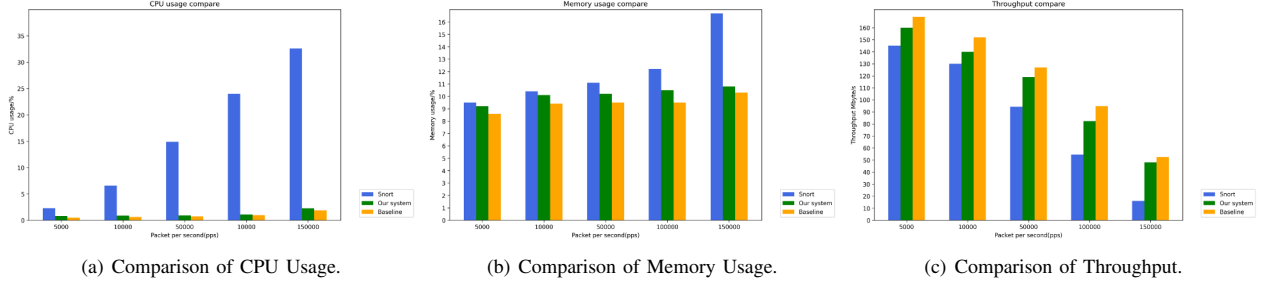


Fig. 8: End-to-End Performance with Different Attack Scales.

TABLE III: Rule List of Snort

Attack Type	Rule
UDP Flood	alert udp any any ->\$EXTERNAL_NET any (msg:"Possible UDP Flood detected"; threshold: type threshold, track by_src, count 100, seconds 10; sid:1000003;)

packets while reducing resource consumption during the process. Experimental results demonstrate that this packet analysis method offers substantial performance enhancements compared to traditional approaches in the context of large-scale network traffic. Additionally, by integrating machine learning techniques, a dynamic blacklist is generated and maintained. This blacklist, based on real-time monitoring of network traffic and behavioral patterns, enables the identification of potential DDoS attacks. Finally, using XDP and longest prefix matching technology, we achieved filtering of abnormal network traffic, thereby enhancing the network's defense capabilities. In the event of an anomaly, the system can automatically mitigate attacks, ensuring the smooth operation of cloud-native applications.

REFERENCES

- [1] G. Pallis, "Cloud computing: the new frontier of internet computing," *IEEE internet computing*, vol. 14, no. 5, pp. 70–73, 2010.
- [2] H. Cloud, "The nist definition of cloud computing," *National institute of science and technology, special publication*, vol. 800, no. 2011, p. 145, 2011.
- [3] D. Gannon, R. Barga, and N. Sundaresan, "Cloud-native applications," *IEEE Cloud Computing*, vol. 4, no. 5, pp. 16–21, 2017.
- [4] C. Benzaïd, M. Boukhalfa, and T. Taleb, "Robust self-protection against application-layer (d) dos attacks in sdn environment," in *2020 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 2020, pp. 1–6.
- [5] A. Javadpour, F. Ja'fari, T. Taleb, and C. Benzaïd, "Reinforcement learning-based slice isolation against ddos attacks in beyond 5g networks," *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 3930–3946, 2023.
- [6] A. Shameli-Sendi, M. Pourzandi, M. Fekih-Ahmed, and M. Cheriet, "Taxonomy of distributed denial of service mitigation approaches for cloud computing," *Journal of Network and Computer Applications*, vol. 58, pp. 165–179, 2015.
- [7] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (ddos) attack dataset and taxonomy," in *2019 international caribbean conference on security technology (ICCST)*. IEEE, 2019, pp. 1–8.
- [8] P. V. Bro, "A system for detecting network intruders in real-time," in *Proc. 7th USENIX security symposium*, 1998.
- [9] M. Roesch *et al.*, "Snort: Lightweight intrusion detection for networks," in *Lisa*, vol. 99, no. 1, 1999, pp. 229–238.
- [10] D. P. D. Kit, "Data plane development kit," 2024, <https://www.dpdn.org>.
- [11] Z. K. Maseer, R. Yusof, N. Bahaman, S. A. Mostafa, and C. F. M. Foozy, "Benchmarking of machine learning for anomaly based intrusion detection systems in the cicsids2017 dataset," *IEEE access*, vol. 9, pp. 22 351–22 370, 2021.
- [12] I. Ullah and Q. H. Mahmoud, "A technique for generating a botnet dataset for anomalous activity detection in iot networks," in *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2020, pp. 134–140.
- [13] M. Gohil and S. Kumar, "Evaluation of classification algorithms for distributed denial of service attack detection," in *2020 IEEE Third International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*. IEEE, 2020, pp. 138–141.
- [14] C.-S. Shieh, W.-W. Lin, T.-T. Nguyen, C.-H. Chen, M.-F. Horng, and D. Miu, "Detection of unknown ddos attacks with deep learning and gaussian mixture model," *Applied Sciences*, vol. 11, no. 11, p. 5213, 2021.
- [15] J. E. Varghese and B. Muniyal, "An efficient ids framework for ddos attacks in sdn environment," *IEEE Access*, vol. 9, pp. 69 680–69 699, 2021.
- [16] C. Zhang and R. Green, "Communication security in internet of thing: preventive measure and avoid ddos attack over iot network," in *Proceedings of the 18th symposium on communications & networking*, 2015, pp. 8–15.
- [17] F. Al-Haidari, M. Sqalli, and K. Salah, "Evaluation of the impact of edos attacks against cloud computing services," *Arabian Journal for Science and Engineering*, vol. 40, pp. 773–785, 2015.
- [18] W. G. Morein, A. Stavrou, D. L. Cook, A. D. Keromytis, V. Misra, and D. Rubenstein, "Using graphic turing tests to counter automated ddos attacks against web servers," in *Proceedings of the 10th ACM conference on Computer and communications security*, 2003, pp. 8–19.
- [19] I. Riadi, R. Umar, and A. Sugandi, "Web forensic on container services using grr rapid response framework," *Scientific Journal of Informatics*, vol. 7, no. 1, pp. 33–42, 2020.
- [20] eBPF, "A thorough introduction to ebpf," 2024, <https://lwn.net/Articles/740157>.
- [21] S. Miano, F. Risso, M. V. Bernal, M. Bertrone, and Y. Lu, "A framework for ebpf-based network functions in an era of microservices," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 133–151, 2021.
- [22] S. Miano, M. Bertrone, F. Risso, M. Tumolo, and M. V. Bernal, "Creating complex network services with ebpf: Experience and lessons learned," in *2018 IEEE 19th International Conference on High Performance Switching and Routing (HPSR)*. IEEE, 2018, pp. 1–8.
- [23] S. Alqahtani and R. F. Gamble, "Ddos attacks in service clouds," in *2015 48th Hawaii International Conference on System Sciences*. IEEE, 2015, pp. 5331–5340.
- [24] P. Szykiewicz, "Signature-based detection of botnet ddos attacks," in *Cybersecurity of Digital Service Chains: Challenges, Methodologies, and Tools*. Springer, 2022, pp. 120–135.